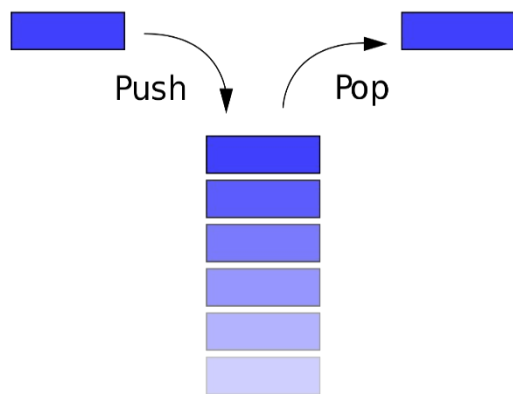


# Stack Data Structures

- **Stack:**

- A Stack is a collection of **Last in First Out (LIFO)** System, is a linear list in which insertion and deletion can take place only one end, called **top** of stack [1]. Examples: A stack of book on Table, A stack of trays or Plates [2, 3].
- Other Names of stacks are “**piles**” and “**push down lists**”.





# Stack Data Structures

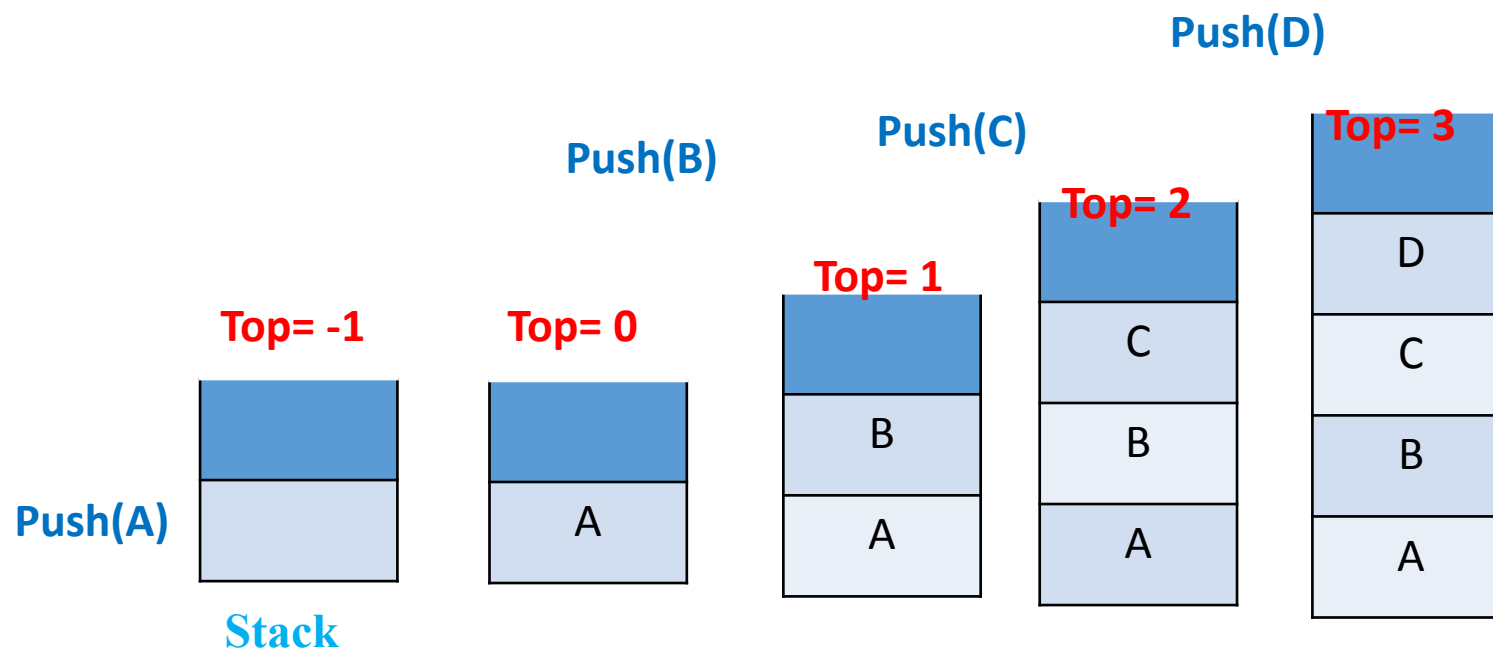
- **Stack:**

- In stack elements are removed from a stack in the reverse order of that in which they were inserted into the stack [1].
- Special Terminology is used for two **basic operations** associated with stacks
  - **Push:** Insert an element into a stack.
  - **Pop:** Delete an element from a stack.

# Stack Data Structures

- **Push Operation:** Insert an element into a stack

- Example:

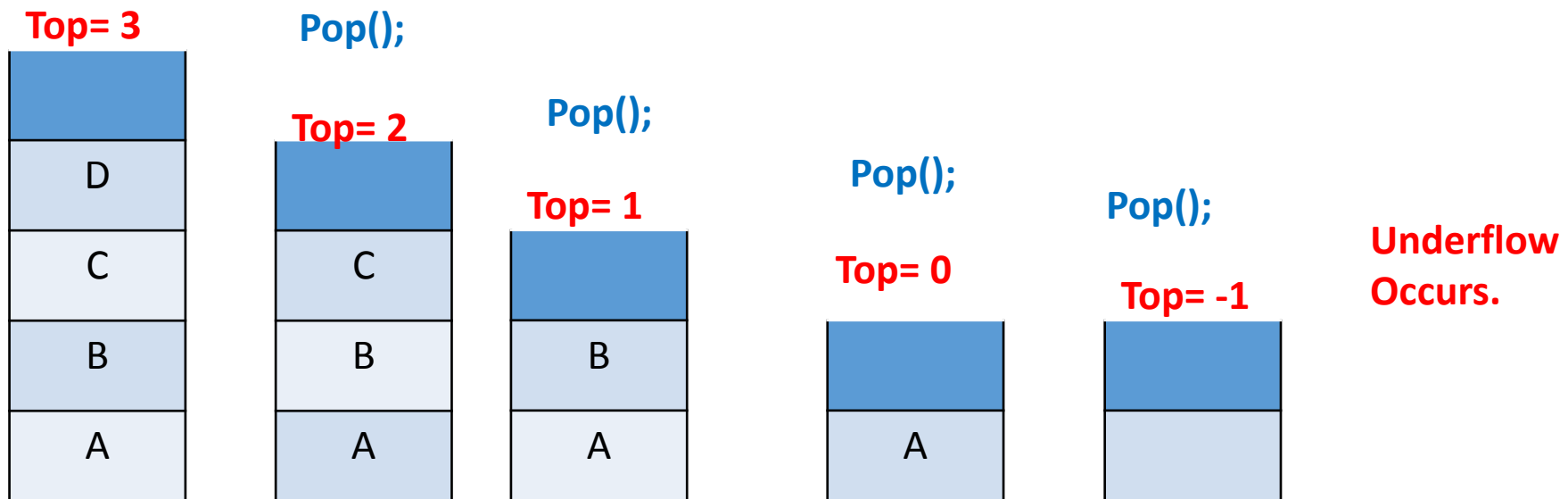


**Overflow Occurs?** If top equal to limit.

**Push(E)**

# Stack Data Structures

- **POP() operation:** Delete an element from a stack
  - Example:





# Stack Data Structures

- **Stack Operations and conditions:**

- **Push:** Insert an element into a stack.
- **Pop:** Delete an element from a stack.
- **Overflow:**
- **Underflow:**
- **Traversing:**
- **Searching:**



# Stack Data Structures

- Array representation of Stack:

```
#define STACKSIZE 100
```

```
struct stack
```

```
{
```

```
    int top;
```

```
    int items[STACKSIZE];
```

```
}
```

```
struct stack s1;
```

```
s1.top=-1;
```



# Stack Data Structures

- Array representation of Stack:

```
#define STACKSIZE 100  
struct stack  
{  
    int top;  
    int items[STACKSIZE];  
}  
struct stack s1;  
s1.top=-1;
```

- Underflow Condition stack(empty):

```
if(s1.top==-1)  
{  
    /* ....Stack is Empty...;  
}  
else  
{  
    /*.....Stack is not empty...;  
}
```



# Stack Data Structures

- **Array representation of Stack:**

```
#define STACKSIZE 100  
struct stack  
{  
    int top;  
    int items[STACKSIZE];  
}  
struct stack s1;  
s1.top=-1;
```

- **Function for Empty Stack:**

```
int empty (struct stack *ps)  
{  
    if( PS □ top == -1)  
    {  
        return (1);  
    }  
    else  
    {  
        return (0);  
    }  
}
```





# Stack Data Structures

- **Array representation of Stack:**

```
#define STACKSIZE 100  
struct stack  
{  
    int top;  
    int items[STACKSIZE];  
}  
struct stack s1;  
s1.top=-1;
```

- **Overflow Condition Stack(Full):**

```
if(s1.top==(STACKSIZE-1))  
{  
    /* ....Stack is Full...;  
}  
else  
{  
    /* .....Stack is not Full...;  
}
```



# Stack Data Structures

- **Array representation of Stack:**

```
#define STACKSIZE 100
struct stack
{
    int top;
    int items[STACKSIZE];
}
struct stack s1;
s1.top=-1;
```

- **Function for Stack is full:**

```
int full (struct stack *ps)
{
    if( PS □ top == (STACKSIZE - 1))
    {
        return (1);
    }
    else
    {
        return (0);
    }
}
```

# Stack Data Structures

- **Push(X) Operation: Insertion**

```
void Push(struct stack *ps, int x)
{
    if (ps->top == (STACKSIZE-1))
    {
        printf ("...Stack is Full....");
    }
    else
    {
        ps->top= ps->top+1;
        ps->items[ps->top]=x;
    }
}
```

```
void Push(struct stack *ps, int x)
{
    if (ps->top == (STACKSIZE-1))
    {
        printf ("...Stack is Full....");
    }
    else
    {
        ps->items[++(ps->top)]=x;
    }
}
```



# Stack Data Structures

- Array representation of Stack:

```
#define STACKSIZE 100

struct stack
{
    int top;
    int items[STACKSIZE];
}

struct stack s1;
s1.top=-1;
```

- Pop() Operation: Deletion

```
int void Pop(struct stack *ps, )
{
    if (empty(ps))
    {
        printf (“...Stack is empty....”);
        return(0);
    }
    else
    {
        return(ps->items[ps->top--]);
    }
}
```



# Stack Data Structures

- Array representation of Stack:

```
#define STACKSIZE 100

struct stack
{
    int top;
    int items[STACKSIZE];
}

struct stack s1;
s1.top=-1;
```

- Stack's Top Element:

```
struct stack s1;
int n;
n= s1.items[s1.top];
```



# Stack Data Structures

- Array representation of Stack:

```
#define STACKSIZE 100

struct stack
{
    int top;
    int items[STACKSIZE];
}

struct stack s1;
s1.top=-1;
```

- Find the number of Elements in the stack?

# Stack Data Structures



- **Applications of Stack Data Structure:**

- Expression Evaluation:
- Syntax Parsing (Compiler):
- Forward and Backward feature of Web Browsers:
- Recursive Problems – Tower of Hanoi, Histograms, Factorial of a number etc.



# Stack Data Structures

- **Difference between Stack and Arrays Data Structure:**

| Stack                                                                                                                                        | Arrays                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• The objects are arranged in a way that they can be added and removed from one end only.</li></ul>    | <ul style="list-style-type: none"><li>• The objects are arranged in a way so that they can be accessed at any time in a random fashion.</li></ul> |
| <ul style="list-style-type: none"><li>• The elements can be added or removed in a LIFO order.</li></ul>                                      | <ul style="list-style-type: none"><li>• It is random access operation. Insertion and deletion takes places in any position.</li></ul>             |
| <ul style="list-style-type: none"><li>• A stack is a dynamic object whose size constantly changing as items are popped and pushed.</li></ul> | <ul style="list-style-type: none"><li>• A arrays may be Static or Dynamic.</li></ul>                                                              |





# Applications of Stack Data Structure

## ❖ Expression Evaluation:

- Example: Infix, Postfix, and Prefix.

$A+B$  ---- Infix Expression,

$+AB$  ----- Prefix Expression

$AB+$  ----- Postfix Expression

## □ Order of Precedence of operators

- Exponential
- Multiplication/Division
- Addition/Subtraction

# Applications of Stack Data Structure

## ◆ Expression Evaluation:

### □ Order of Precedence of operators

| Precedence | Operator                                   | Associativity     |
|------------|--------------------------------------------|-------------------|
| 1          | (), []                                     | non-associative   |
| 2          | new                                        | non-associative   |
| 3          | .                                          | left-associative  |
| 4          | ++, --                                     | non-associative   |
| 5          | - (unary), + (unary), !, ~, ++, --, (type) | right-associative |
| 6          | *, /, %                                    | left-associative  |
| 7          | +, -                                       | left-associative  |



# Applications of Stack Data Structure

## ◆ Expression Evaluation:

### □ Order of Precedence of operators

|    |                                               |                   |
|----|-----------------------------------------------|-------------------|
| 8  | <<, >>, >>>                                   | left-associative  |
| 9  | <, >, <=, >=, instanceof                      | non-associative   |
| 10 | ==, !=                                        | left-associative  |
| 11 | &                                             | left-associative  |
| 12 | ^                                             | left-associative  |
| 13 |                                               | left-associative  |
| 14 | &&                                            | left-associative  |
| 15 |                                               | left-associative  |
| 16 | ?:                                            | right-associative |
| 17 | =, *=, /=, %=, -=, <<=, >>=, >>>=, &=, ^=,  = | right-associative |



# Applications of Stack Data Structure

## ❖ Expression Conversion Examples:

### ❖ Example 1.- $A+B*C$

step1:  $A+BC^*$  or  $A+(BC^*)$  -----  $*$  have more precedence

step2:  $ABC^* +$

### ❖ Example 2. $(A+B)*(C-D)$

step1:  $AB+*(C-D)$

step2:  $AB+*CD-$

step3:  $AB+CD-*$



# Applications of Stack Data Structure

## ◆ Expression Conversion Examples:

### ◆ Example 3:

$A\$B * C - D + E / F / (G + H)$      suppose the \$ have higher precedence to \*, /, +, -.

Step1: A \$ B \* C - D + E / F / G H +

Step2: A B \$ \* C - D + E / F / G H +

Step3: A B \$ C \* - D + E / F / G H +

Step4: A B \$ C \* - D + E F / / G H +

Step5: A B \$ C \* - D + E F / G H + /

Step6: A B \$ C \* D - + E F / G H + /

Step7: A B \$ C \* D - E F / G H + / +



# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

◆  $6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ +\ *\ 2\ \$\ 3\ +$



**Stack**



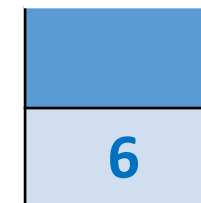
# Applications of Stack Data Structure

## ❖ Evaluating a Postfix Expression:

### ❖ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |



**Stack**

- ❖ Operand will come push into stack
- ❖ Operator will come pop two times and calculate value and push that value



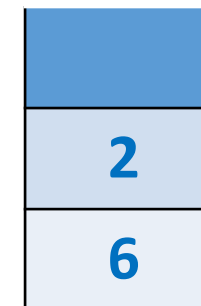
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |



**Stack**

- ❖ Operand will come push into stack
- ❖ Operator will come pop two times and calculate value and push that value





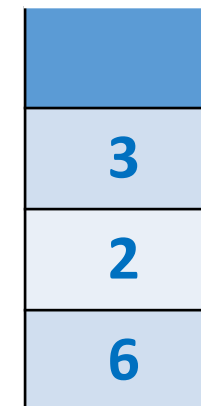
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

◆ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |
| 3      | -         | -         | -     |



Stack

# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |
| 3      | -         | -         | -     |
| +      | 2         | 3         |       |



**Stack**

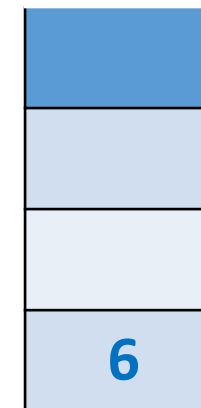
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |
| 3      | -         | -         | -     |
| +      | 2         | 3         | 5     |



**Stack**



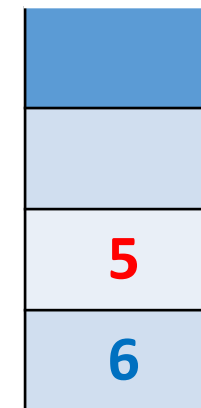
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |
| 3      | -         | -         | -     |
| +      | 2         | 3         | 5     |



Stack

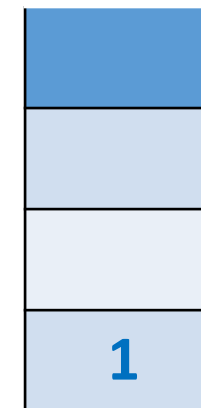
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |
| 3      | -         | -         | -     |
| +      | 2         | 3         | 5     |
| -      | 6         | 5         | 1     |



**Stack**

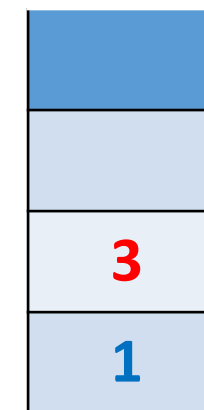
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 6      | --        | --        | --    |
| 2      | --        | --        | --    |
| 3      | -         | -         | -     |
| +      | 2         | 3         | 5     |
| -      | 6         | 5         | 1     |
| 3      | -         | -         | -     |



**Stack**



# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 3      | -         | -         | -     |
| 8      | -         | -         | -     |



Stack

# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 3      | -         | -         | -     |
| 8      | -         | -         | -     |
| 2      | -         | -         | -     |



**Stack**





# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 3      | -         | -         | -     |
| 8      | -         | -         | -     |
| 2      | -         | -         | -     |
| /      | 8         | 2         | 4     |



**Stack**



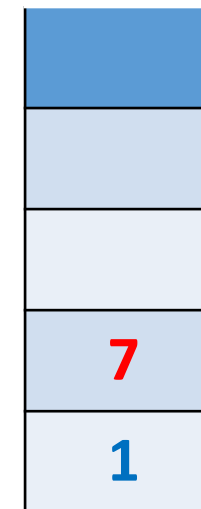
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 3      | -         | -         | -     |
| 8      | -         | -         | -     |
| 2      | -         | -         | -     |
| /      | 8         | 2         | 4     |
| +      | 3         | 4         | 7     |



Stack



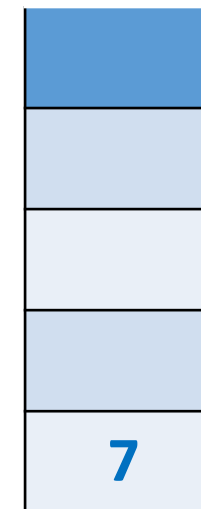
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

◆ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| /      | 8         | 2         | 4     |
| +      | 3         | 4         | 7     |
| *      | 1         | 7         | 7     |



**Stack**

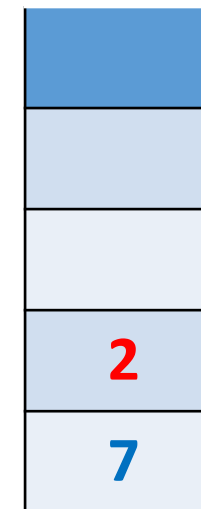
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| /      | 8         | 2         | 4     |
| +      | 3         | 4         | 7     |
| *      | 1         | 7         | 7     |
| 2      | -         | -         | -     |



**Stack**



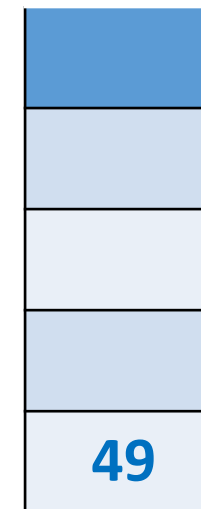
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| /      | 8         | 2         | 4     |
| +      | 3         | 4         | 7     |
| *      | 1         | 7         | 7     |
| 2      | -         | -         | -     |
| \$     | 7         | 2         | 49    |



**Stack**

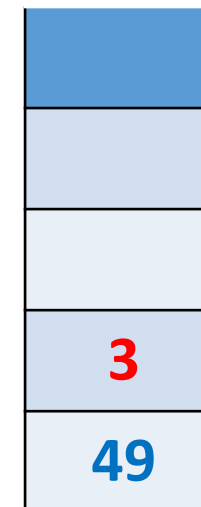
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 2      | -         | -         | -     |
| \$     | 7         | 2         | 49    |
| 3      | -         | -         | -     |



**Stack**

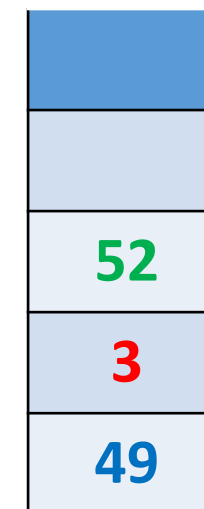
# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### ◆ Example:

❖ 6 2 3 + - 3 8 2 / + \* 2 \$ 3 +

| Symbol | operand 1 | operand 2 | Value |
|--------|-----------|-----------|-------|
| 2      | -         | -         | -     |
| \$     | 7         | 2         | 49    |
| 3      | -         | -         | -     |
| +      | 49        | 3         | 52    |



**Stack**



# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

### Algorithm:

opndstk = the empty stack:

// scan the input string reading one by one character

// element at a time into symb \*/

While (not end of the input)

{

    symb= next input character;

    if (symb is an operand)

        {

            push(openstk, symb);

        }





# Applications of Stack Data Structure

## ◆ Evaluating a Postfix Expression:

Algorithm:

```
.... else
    {
        Opnd2= pop(opndstk);
        Opnd1= pop(opndstk);
        Value = result of applying symbol to open1 and open2
        push(opndstk, value)
    }
} rerurn( pop(opndstk));
```

# Problems Based on Arrays

- ❖ Write a C program to count bits set in an integer [2]?
  - ❖ There are a number of ways to count the number of bits set in an integer. Here are some C programs to do the same.

- ❖ **Method1:**

```
#include<stdio.h>
int main()
{
    unsigned int num=10;
    int ctr=0;

    for(;num!=0;num>>=1)
    {
        if(num&1)
        {
            ctr++;
        }
    }

    printf("\n Number of bits set in [%d] = [%d]\n", num, ctr);
    return(0);
}
```





# Problems Based on Arrays

## ❖ Write a C program to count bits set in an integer [2]?

- ❖ **Method 2:** This is a faster way of doing the same thing. Here the control goes into the while loop only as many times as the number of bits set to 1 in the integer!.

```
#include<stdio.h>
int main()
{
    int num=10;
    int ctr=0;

    while (num)
    {
        ctr++;
        num = num & (num - 1); // This clears the least significant bit set.
    }

    printf("\n Number of bits set in [%d] = [%d]\n", num, ctr);
    return(0);
}
```



# Problems Based on Arrays

❖ Write a C program to count bits set in an integer [1]?

❖ **Method 3:** This method is very popular because it uses a lookup table. This speeds up the computation. What it does is it keeps a table which hardcodes the number of bits set in each integer from 0 to 256.

```
0 - 0 Bit(s) set.  
1 - 1 Bit(s) set.  
2 - 1 Bit(s) set.  
3 - 2 Bit(s) set.  
...
```

# Problems Based on Arrays

❖ Write a C program to count bits set in an integer [1]?

```
const unsigned char LookupTable[] = { 0, 1, 1, 2, 1, 2, 2, 3, 1, 2, 2, 3, 2, 3, 3, 4,
    1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
    1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
    2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
    1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
    2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
    2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
    3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
    1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
    2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
    2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
    3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
    2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
    3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
    3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
    4, 5, 5, 6, 5, 6, 6, 7, 5, 6, 6, 7, 6, 7, 7, 8};
```

```
unsigned int num;
unsigned int ctr;    // Number of bits set.

ctr = LookupTable[num & 0xff] +
    LookupTable[(num >> 8) & 0xff] +
    LookupTable[(num >> 16) & 0xff] +
    LookupTable[(num >> 24)];
```



# Applications of Stack Data Structure

## • Problem 1: Poisonous Plants Problem

- ❖ There are a number of **plants in a garden**. Each of these plants has been **treated with some amount of pesticide [4]**.
- ❖ **After each day**, if any **plant** has more pesticide than the plant on its left, being **weaker than the left one, it dies**. **Example: [4, 5] then 4 will dies**.
- ❖ You are given the initial values of the pesticide in each of the plants. Print the number of days after which no plant dies



# Applications of Stack Data Structure

## ❖ Problem 1:

- ❖ For example, pesticide levels  $PES=[13,26,12,17,15, 34, 66,42, 32, 11]$ .
  - ❖ After **day 1** plants (13, 12, 15, 34) die.
  - ❖ Leaving  $p=[26, 17, 66, 42, 32, 11]$ .
- ❖ On day 2, plant 17 of the current array dies.
- ❖ Leaving  $p=[26, 66, 42, 32, 11]$ .



# Applications of Stack Data Structure

## ❖ Problem 1:

- ❖ For example, pesticide levels  $PES=[13,26,12,17,15, 34, 66,42, 32, 11]$ .
  - ❖ Leaving  $p=[26, 66, 42, 32, 11]$ .
  - ❖ On day 3, plant 26 of the current array dies.
  - ❖ Leaving  $p=[66, 42, 32, 11]$ .
- ❖ As there is no plant with a higher concentration of pesticide than the one to its left, plants stop dying after day 3.



# Applications of Stack Data Structure

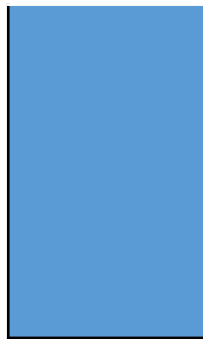
---



**Problem 2:** Stacks minimum element in a constant time  $O(1)$

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ **Solution:** Using the another stack.
  - ❖ Another stack is used to keep the minimum at top
  - ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2



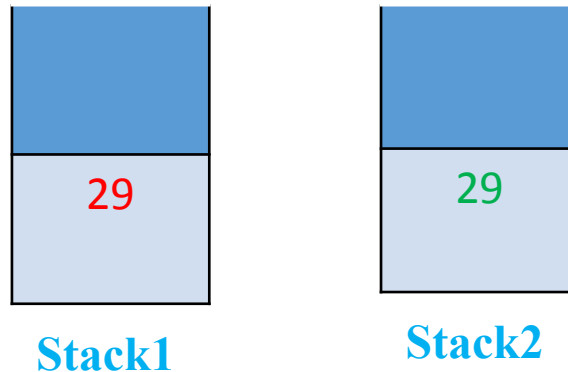
Stack 1



Stack 2

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ Minimum will be Stack2(top).

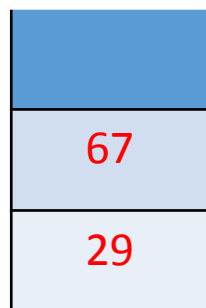


## ❖ Push function

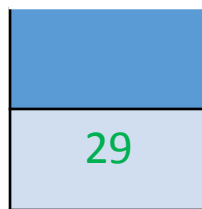
```
if(s1.top==-1 and s2.top==-1)
{
    s1.top++, s2.top++;
    stack1(push(x));
    stack2(push(x));
}
```

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ Minimum will be Stack2(top).



Stack1



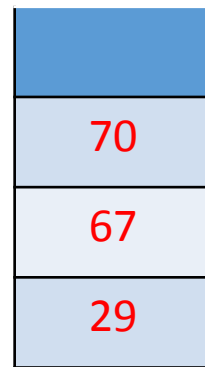
Stack2

- ❖ Push function

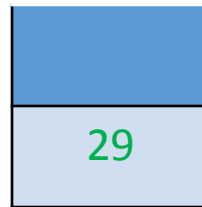
```
elseif( x >= s2.top)
{
    stack1(push(x));
}
```

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ Minimum will be Stack2(top).



Stack1



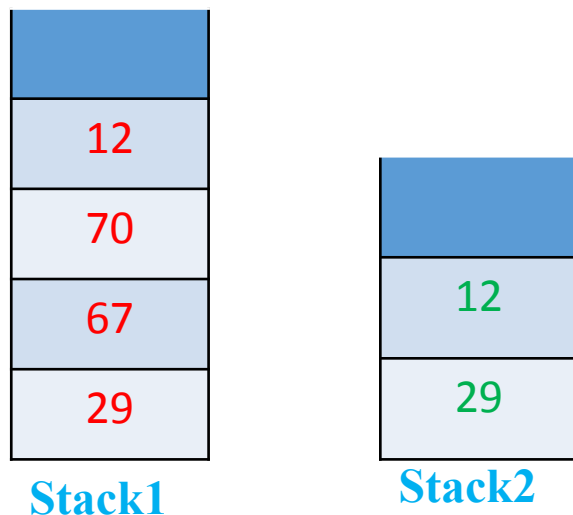
Stack2

- ❖ Push function

```
elseif( x >= s2.top)
{
    stack1(push(x));
}
```

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ X = element to be pushed, Minimum will be Stack2(top).

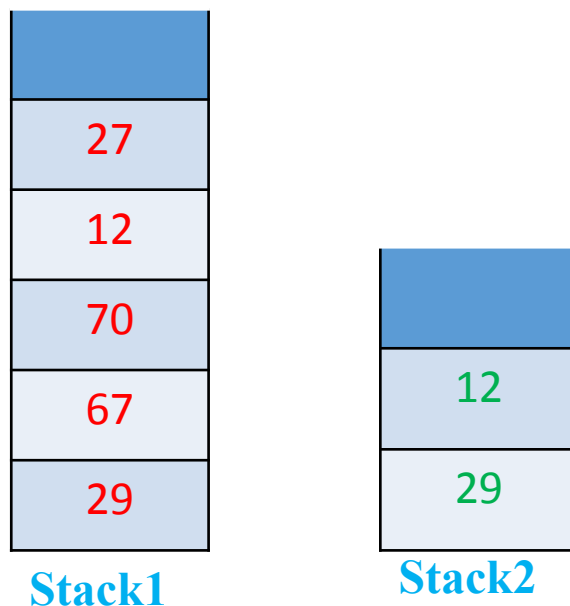


- ❖ Push function

```
elseif( x < s2.top)
{
    stack1(push(x));
    stack2(push(x));
}
```

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ Minimum will be Stack2(top).

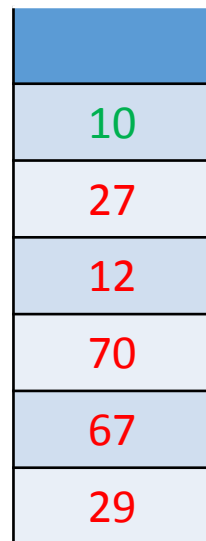


- ❖ Push function

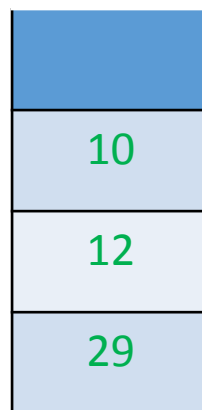
```
elseif( x >= s2.top)
{
    stack1(push(x));
}
```

# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ Minimum will be Stack2(top).



Stack1



Stack2

- ❖ Push function

```
elseif( x < s2.top)
{
    stack1(push(x));
    stack2(push(x));
}
```

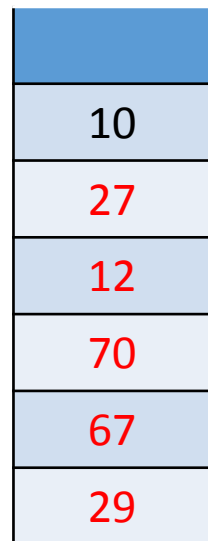


# Applications of Stack Data Structure

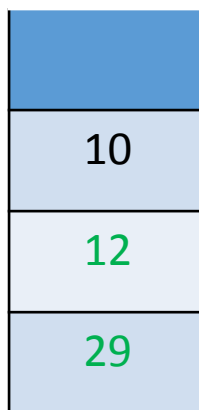
❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$

❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2

❖ Minimum will be Stack2(top). PoP()



Stack1



Stack2

❖ Pop function

if( s2.top== -1 || s1. top== -1)

{ stack empty;

}

elseif(s1.top == s2.top)

{ stack1(pop());

stack2(pop());

}

# Applications of Stack Data Structure

❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$

❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2

❖ Minimum will be Stack2(top). PoP()

❖ Pop function

```
if( s2.top== -1 || s1.top== -1)
```

```
{    stack empty;
```

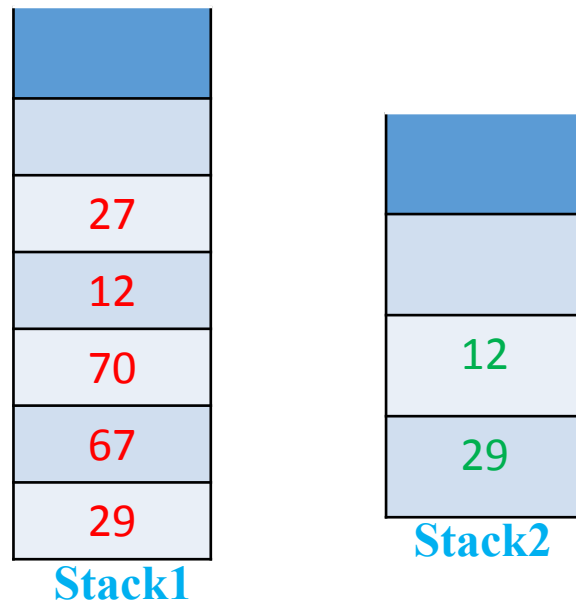
```
}
```

```
elseif(s1.top == s2.top)
```

```
{    stack1(pop());
```

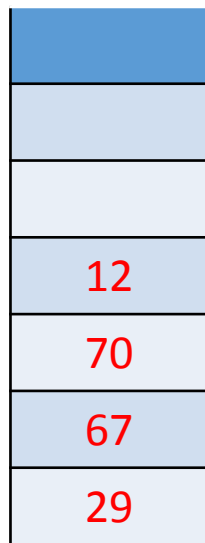
```
    stack2(pop());
```

```
}
```

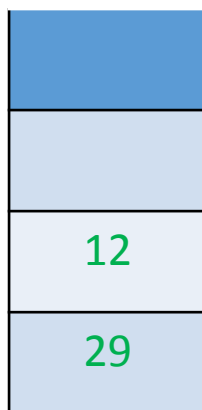


# Applications of Stack Data Structure

- ❖ **Problem 2:** Stacks minimum element in a constant time  $O(1)$
- ❖ Example: 29, 67, 70, 12, 27, 10, 20, 7, 5, 19, 26, 3, 35, 2
  - ❖ Minimum will be Stack2(top). PoP()



Stack1



Stack2

- ❖ Pop function

```
if( s2.top== -1 || s1.top== -1)
{
    stack empty;
}
elseif(s1.top> s2.top)
{
    stack1(pop());
}
```



# Assignment-1

---

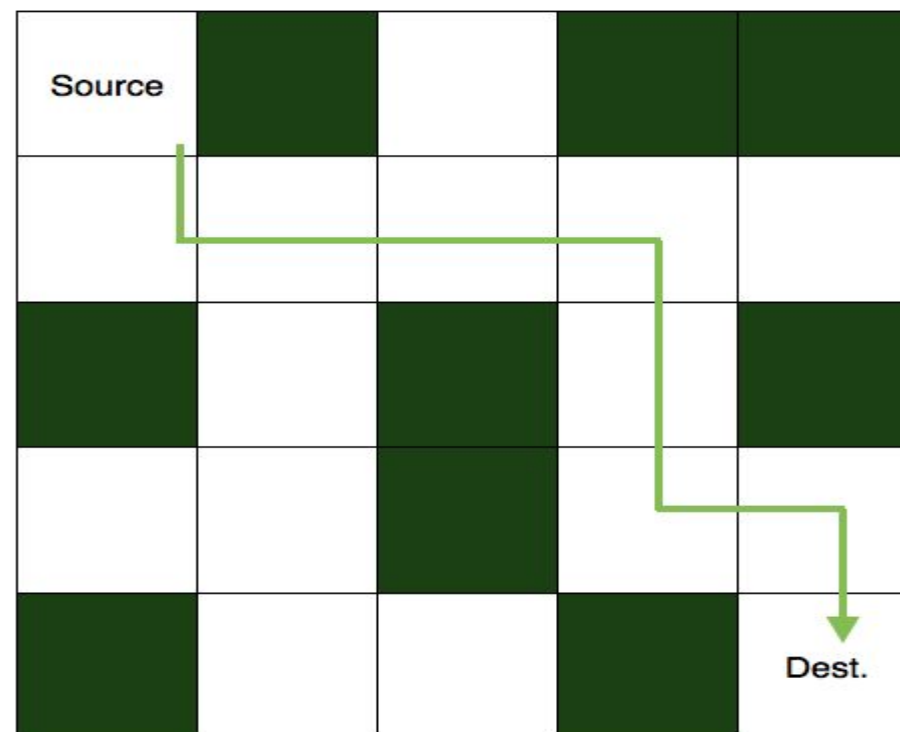
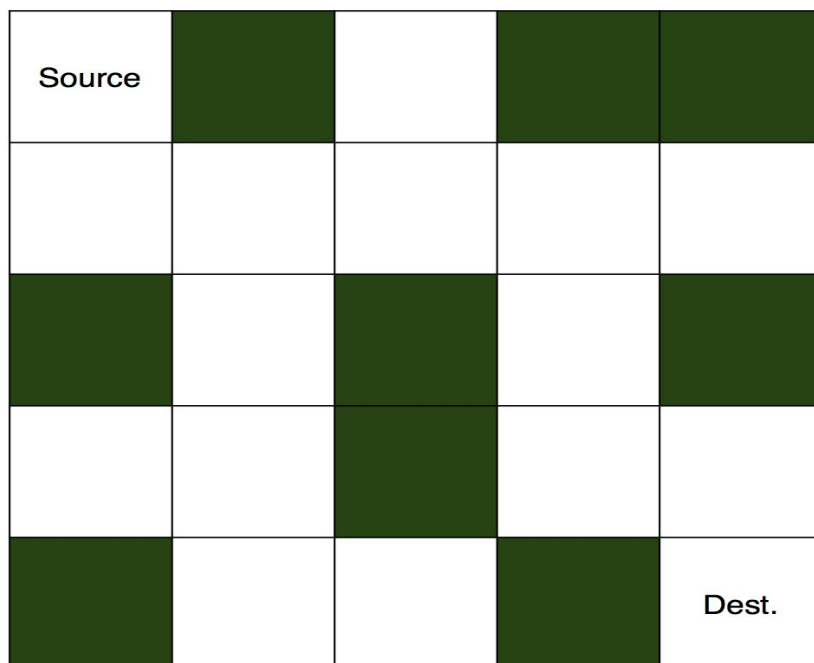
**Problem-1:** The Rat In A Maze problem ?

**Problem-2:** Matching Nuts & Bolts Problem (Lock & Key problem)

- Nuts represented as array of character  
`char nuts[] = {'@', '#', '$', '%', '^', '&', '*', '+'}`
- Bolts represented as array of character  
`char bolts[] = {'$', '%', '+', '&', '*', '^', '@', '#'}`

# Assignment-1

❖ Problem: The Rat In A Maze problem ?





# References

1. Seymour Lipschutz, “Data Structures”, Schaum’s Series McGraw Hill edition 2013.
2. [https://miro.medium.com/max/957/1\\*VWK5OyJIL5Cyg0Hu8dLiXQ.jpeg](https://miro.medium.com/max/957/1*VWK5OyJIL5Cyg0Hu8dLiXQ.jpeg)
3. <https://en.wikipedia.org/>